ÜBERBLICK

# Überblick und Gesamtaufbau

Einordnung, Geräteaufbau, Rechen- und Leitwerk, Befehlsphasen, Speicher-Timing und Teilprojekte des C 8205

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

| | | |
|---|---|---|
| Quelle | C8205-Interpreter-Überblick_und_Gesamtaufbau.md | 0.95-009 |
| Stand | 10.11.2026 | |
| Umfang | 11 Seiten, A4 hoch | |
| Erzeugt mit | erzeuge_ueberblick_gesamtaufbau_pdf.py | |

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

# C 8205 – Überblick und Gesamtaufbau

Quelle: C8205-Interpreter-Überblick_und_Gesamtaufbau.md · im Wortlaut der Quelle

## Inhaltsverzeichnis


**1. Einordnung und Leistungsdaten . . . 3**

**2. Geräteaufbau . . . 3**

**3. Rechen- und Leitwerk . . . 4**

**4. Ablauf der Befehlsabarbeitung – die vier Phasen . . . 5**

4.1 Informationszustand nach den Phasen . . . 6

4.2 Besonderheiten im Phasenablauf . . . 7

**5. Speicher-Timing und Leistungswerte . . . 7**

5.1 Zeitoptimale Programmierung . . . 8

**6. Teilprojekte . . . 9**

6.1 Interpreter . . . 9

6.2 Assembler . . . 10

6.3 BIS (Bausteininterpretiersystem) . . . 10

**7. Verweise auf die Teildokumente . . . 10**

**8. Offene Punkte . . . 11**

# C 8205 – Überblick und Gesamtaufbau



Version: 0.95-009

Dieses Dokument fasst die allgemeinen, einführenden Angaben zum C 8205 zusammen, wie sie in den Übersichtsgrafiken (upload_c8205-1 … upload_c8205-6) enthalten sind, und ordnet die Teilprojekte (Interpreter, Assembler, BIS) ein. Details, die in Teildokumenten ausführlich beschrieben sind, werden hier nicht wiederholt, sondern nur referenziert (Abschnitt 7).

Die Verzeichnisstruktur des Gesamtprojekts (welcher Pfad was enthält) steht nicht hier, sondern im Wurzeldokument "C8205-Projektstruktur.md".

Das Projekt hat drei Teilprojekte:

- den **Interpreter** (Python): führt Programme des C 8205 interpretativ nach dem Knotengraphen aus (Abschnitt 6.1),
- den **Assembler** `c82ass` (C): übersetzt Assembler-Quelltext in Objektcode für den C 8205 (Abschnitt 6.2),
- das **Bausteininterpretiersystem BIS**: Programmsystem (Anwendungs-Software), das auf dem C 8205 selbst läuft (Abschnitt 6.3).

## 1. Einordnung und Leistungsdaten

Der C 8205 ist ein **Einadreßautomat**. Bei zeitoptimaler Programmierung lassen sich etwa 2000 verdrahtete Operationen je Sekunde ausführen.

Als Vorzüge werden genannt:

- geringer technischer Aufwand der Zentraleinheit (nur ca. 400 Transistoren und 2000 Dioden),
- die Magnettrommel als vergleichsweise billiges Speichermedium mit relativ kleiner Zugriffszeit,
- für einen Kleinrechner hohe Operationsgeschwindigkeit,
- Programmierhilfen in Form sogenannter Interpretersysteme, die den Programmieraufwand wesentlich verringern.

Damit ist der C 8205 ein für die eingesetzte Technik leistungsstarker Kleinrechenautomat, einsetzbar

- für wissenschaftlich-technische Berechnungen,
- für mathematisch-ökonomische Berechnungen,
- zur Datenverarbeitung in begrenztem Umfang,
- als Hilfs- oder Satellitenrechner im Rahmen eines Systems von Rechnern oder Datenverarbeitungsanlagen.

(Der Begriff "Interpretersystem" in dieser historischen Aufzählung meint Programmiersysteme auf dem C 8205 selbst; er ist nicht zu verwechseln mit dem Python-Interpreter dieses Projekts, Abschnitt 6.1.)

## 2. Geräteaufbau

Der Rechner besteht aus:

- der **Stromversorgungseinheit** (Anschlussspannung 220 V ± 10 %, 50 ± 1 Hz, Leistungsaufnahme rund 1500 VA),
- der **Zentraleinheit**, bestehend aus Rechenwerk und Leitwerk.

Zur unmittelbar zugeordneten (1.) Peripherie gehören:

- der Magnettrommelspeicher (Organisation und Kapazität: siehe Dokument "C8205-Interpreter-Magnettrommel.md"),
- Tastatur, Lampenfeld und Bahnsperrenschalter (Bedienelemente: siehe Dokument "C8205-Interpreter-Bedienpult_mit_Schaltern.md"),
- 2 Lochbandleser (Leser 1 an Kanal K1, Leser 2 an Kanal K2; Eingabegeschwindigkeit des Standardlesers 303 130 Zeichen/s, des Lesers FS751 750 Zeichen/s (GRAFIK-Schalter `--LS-LESER-TYP`); Zeichen im 8-Kanal- oder 5-Kanal-Kode; Details: "C8205-Interpreter-Lochstreifenleser.md"),
- 1 Lochbandstanzer (maximale Stanzgeschwindigkeit 50 Zeichen/s, Kanal K1),
- 1 Schreibwerk für alphanumerische Ein- und Ausgabe (maximale Schreibgeschwindigkeit 10 Zeichen/s; Eingabe über Kanal K0, Ausgabe über Kanal K2).

Der Informationsfluss zwischen diesen Baueinheiten: Pult-Tastatur sowie die beiden Lochbandleser speisen (jeweils über einen Puffer) das Leitwerk/Rechenwerk, das seinerseits mit dem Trommelspeicher verbunden ist; vom Leitwerk/Rechenwerk führt der Weg über einen weiteren Puffer zu Schreibwerk und Lochbandstanzer.

Abbildung 1 zeigt die Baueinheiten auf ihren Tischen: links den Peripherietisch mit Lochbandstanzer und den beiden Lesern (Leser 2 links von Leser 1), in der Mitte den Rechentisch mit dem Bedienpult, rechts das Schreibwerk auf dem Schreibwerktisch.

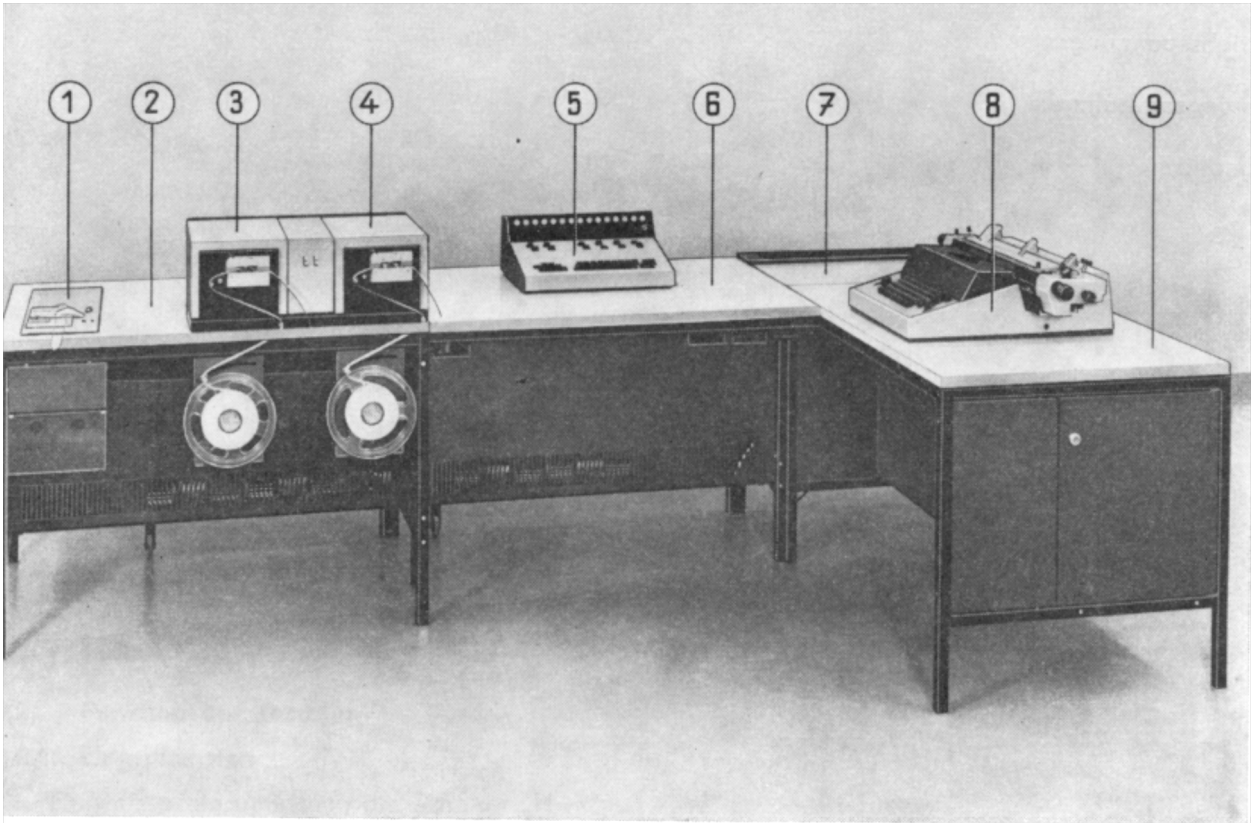



Abbildung 1: Gesamtansicht des C 8205 (Foto, Originalunterlagen). 1 Lochbandstanzer, 2 Peripherietisch, 3 Leser 2, 4 Leser 1, 5 Tastatur-Anzeige-Einheit (Bedienpult), 6 Rechentisch, 7 SVE-Tisch (Stromversorgungstisch), 8 Ein- und Ausgabe-Schreibwerk, 9 Schreibwerktisch

Als **2. Peripherie** (wechselnd anschließbar) kommen lochbanderzeugende bzw. lochbandverarbeitende Geräte in einem beliebigen 5-Kanal- oder 8-Kanal-Kode in Frage (beim 8-Kanal-Kode mit Paritätslochung im 5. Kanal). Programmsysteme werden meist im 8-Kanal-R300-Kode oder im 5-Kanal-CCITT-2-Kode (auch Telegraphenalphabet Nr. 2 genannt) geliefert. Die Zeichenkode-Tabelle (R300-Lochstreifenkode, Tabelle 1) steht in "C8205-Interpreter-Kodetabellen.md".

## 3. Rechen- und Leitwerk

Das **Leitwerk** besitzt den Befehlszähler (BZ), das Adreßregister (AR) und das Befehlsregister (BR). Das Befehlsregister (21 Bit) besteht aus dem Adreßteil a (12 Bit) und dem Operationsregister OR (9 Bit); das Adreßregister AR (12 Bit) ist davon getrennt der Zellenwähler für den Trommelzugriff. (Bit-Breiten und Feldaufteilung: siehe Dokument "C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md".)

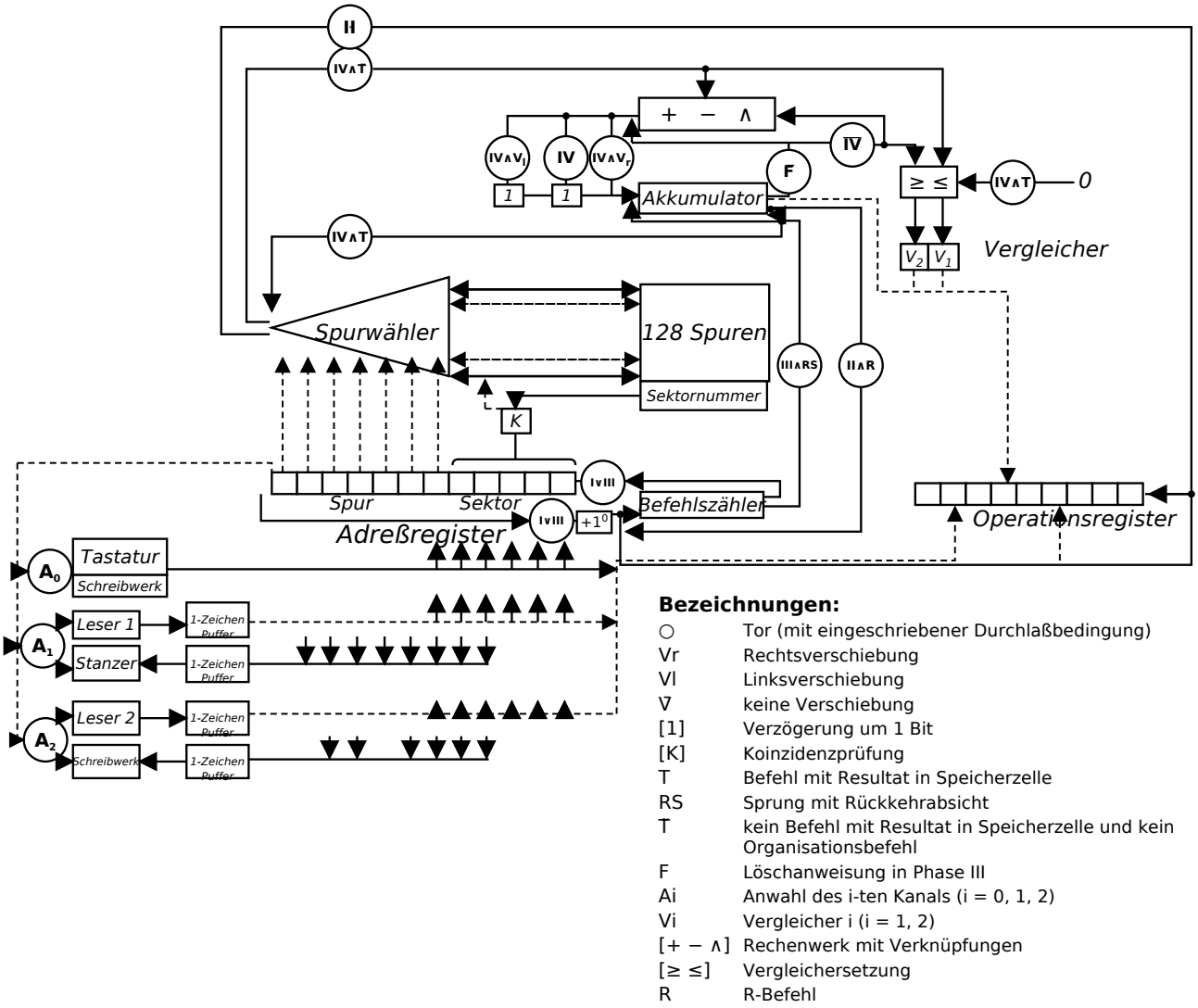
Zum **Rechenwerk** gehören der Akkumulator (AC) und das Rechenwerk im engeren Sinn; dieses kann außer Addition und Subtraktion auch die logische UND-Verknüpfung (Konjunktion) ausführen.

Die Register unterscheiden sich in ihrer Bauart:

- **BZ und AC sind Umlaufregister**, die sich technisch auf der Trommel befinden (Umlaufspeicherprinzip): Ein Lesekopf (H) erfasst die Bit-Belegung eines Platzes, ein Umformer (U) bereitet darauf ein erneutes Aufsprechen über einen Sprechkopf (S) vor, ein Löschkopf (L) löscht die Plätze (setzt sie auf 0). Die zentrale Taktsteuerung sorgt dafür, dass der Informationszustand eines solchen Umlaufregisters zu Beginn jeder Wortzeit stets derselbe ist.
- **OR (9 Dualstellen) und AR (12 Stellen) sind dagegen statische Register**, als Schiebekette aufgebaut: Die Information wird seriell ein- und herausgeschoben; der Zustand jeder Stelle ist zu jedem Zeitpunkt abgreifbar. Im AR geschieht dies für die Spur- und die Sektor-kennzeichnenden Stellen getrennt.

Zugriffsrichtung zum Trommelspeicher: Eine **Entnahme** gespeicherter Information ist zum Rechenwerk, zum Befehlszähler und zum Operationsregister hin möglich. Eine **Abspeicherung** von Information im Speicher ist dagegen nur vom Akkumulator her möglich.

Abbildung 2 zeigt das Blockschaltbild: Akkumulator, Rechenwerk und Vergleicher, Spurwähler und Trommel, Adreßregister, Befehlszähler und Operationsregister sowie die Kanäle zu Tastatur, Schreibwerk, Lochbandlesern und Stanzer, verbunden durch Leitungen mit Toren.




**Bezeichnungen:**

| | |
|---|---|
| ○ | Tor (mit eingeschriebener Durchlaßbedingung) |
| Vr | Rechtsverschiebung |
| Vl | Linksverschiebung |
| V̄ | keine Verschiebung |
| [1] | Verzögerung um 1 Bit |
| [K] | Koinzidenzprüfung |
| T | Befehl mit Resultat in Speicherzelle |
| RS | Sprung mit Rückkehrabsicht |
| T̄ | kein Befehl mit Resultat in Speicherzelle und kein Organisationsbefehl |
| F | Löschanweisung in Phase III |
| Ai | Anwahl des i-ten Kanals (i = 0, 1, 2) |
| Vi | Vergleicher i (i = 1, 2) |
| [+ − ∧] | Rechenwerk mit Verknüpfungen |
| [≥ ≤] | Vergleichersetzung |
| R | R-Befehl |

Abbildung 2: Blockschaltbild des C 8205 (Nachzeichnung). Die Legende „Bezeichnungen“ steht in der Abbildung.

**Lesehilfe:** Ein Kreis auf einer Leitung ist ein **Tor**; die Beschriftung nennt die Durchlaßbedingung, unter der das Tor Information durchläßt. Römische Ziffern bedeuten die Phase des Befehlszyklus (Abschnitt 4), „∧“ ein gleichzeitiges Zutreffen, „∨“ ein alternatives. Die Zusätze nach der Legende der Abbildung: T = Befehl mit Resultat in Speicherzelle, T̄ = kein solcher Befehl und kein Organisationsbefehl, RS = Sprung mit Rückkehrabsicht, R = R-Befehl, Vr = Rechtsverschiebung, Vl = Linksverschiebung, V̄ = keine Verschiebung; F bezieht sich auf die Löschanweisung in Phase III. Die Tore A0, A1, A2 wählen den i-ten Kanal an. Gestrichelte Leitungen und Pfeilrichtungen zeigen die Datenwege; [K] bedeutet Koinzidenzprüfung, [1] Verzögerung um 1 Bit.

## 4. Ablauf der Befehlsabarbeitung – die vier Phasen

Charakteristisch für den Einadreßautomaten C 8205 ist, dass als Zellenwähler in jedem Fall das Adreßregister AR benutzt wird, und dass bei Entnahmen aus dem Speicher – wegen der Verwendung einer Magnettrommel als Speicher – im Allgemeinen Wartezeiten auftreten, da der gewünschte Sektor erst unter die Lese-Schreib-Einrichtung gedreht werden muss.

BZ enthält dabei die Adresse BADR des nächstabzuarbeitenden Befehls.

# C 8205 – Überblick und Gesamtaufbau

Quelle: C8205-Interpreter-Überblick_und_Gesamtaufbau.md · im Wortlaut der Quelle

Der Befehlszyklus gliedert sich in vier Phasen, zwischen denen jeweils eine Koinzidenz-Wartezeit liegt. Die folgende Tabelle beschreibt einen **vergröberten Prinzip-Ablauf**: Sie benennt je Phase nur die dafür charakteristische Hauptwirkung. Einzelne, hier nicht gesondert aufgeführte Nebeneffekte (z. B. weitere, in der Elektronik gleichzeitig ablaufende Registerbelegungen) können davon abweichend auch an anderer Stelle im Zyklus auftreten; das ist keine Inkonsistenz, sondern Folge dieser vereinfachten Darstellungstiefe.

| Phase | Bezeichnung | Wirkung |
|---|---|---|
| I | Befehlszähleraufstellung | Füllen des Zellenwählers AR mit BADR: `<AR> := <BZ> = BADR` |
| (K) | $W_1$ — Wartezeit | Trommel dreht sich bis zum Sektor, der durch `<AR> = BADR` vorgegeben ist |
| II | Befehlsentnahme und Operationsaufstellung | Entnahme des Befehls ADR OP: `<BZ> := ADR; <OR> := OP` |
| III | Befehlsadressenaufstellung | Füllen des Zellenwählers AR mit ADR, gleichzeitig Rücktransport von BADR nach BZ und Erhöhung um 1: `<AR> := <BZ> = ADR`, gleichzeitig `<BZ> := <AR>+1 = BADR+1` |
| (K) | $W_2$ — Wartezeit | Trommel dreht sich bis zum Sektor, der durch `<AR> = ADR` vorgegeben ist |
| IV | Operationsausführung | Ausführung entsprechend `<OR> = OP` |

Im Blockschaltbild (Abbildung 2, Abschnitt 3) stehen die Phasen als Durchlaßbedingungen an den Toren; so ist etwa das Tor „III ∧ RS“ in Phase III beim Sprung mit Rückkehrabsicht durchlässig, und die beiden Tore am Adreßregister („I ∨ III“) füllen den Zellenwähler in den Phasen I und III.

Zwischen Phase III und Phase IV kann von außen eingegriffen werden (AC-Löschung, Stop, Start) – die Bedeutung der zugehörigen Tasten (E, G, M, Ü, ST) und des Schalters H-R ist im Dokument "C8205-Interpreter-Bedienpult_mit_Schaltern.md" beschrieben.

Eine Phase benötigt 104 µs, d. h. eine Wortzeit.

Diese vier Phasen entsprechen den "Phasenzuordnung"-Knoten der Ablaufbeschreibung (Knoten 2, 8, 15 und 31/61/67 für Beginn Phase I–IV, siehe Dokument "C8205-Interpreter-Eingänge_der_Knoten.md").

**Interpretative Abarbeitung:** Der Python-Interpreter (Abschnitt 6.1) rechnet mit Wortzeit und Wartezeit = 0; die Trommeldrehung wird nicht nachgebildet (Ausnahme: die Befehlsentnahme nach einem X-Abbruch, Abschnitt 4.1). Die Zeitangaben dieses Abschnitts und des Abschnitts 5 beschreiben die Hardware, nicht den Interpreter.

## 4.1 Informationszustand nach den Phasen

Die Programmieranleitung (Abschnitt 2.2 "Befehlsschleife", Seitenabbild) gibt für jede Phase den Zustand von BZ, AR, OR und AC an. Bezeichnungen: b = Adresse des abzuarbeitenden Befehls, a = sein Adreßteil, ω = sein Operationsteil, ω* = Operationsteil nach einer eventuellen Umstellung in Phase II (R-Befehle, bedingte Sprünge mit erfüllter Bedingung), $R_{-1}$ = Resultat des vorangegangenen Befehls, R = Resultat dieses Befehls. Vor Phase I steht im BZ die Adresse b des nächsten Befehls.

| nach Phase | BZ | AR | OR | AC |
|---|---|---|---|---|
| I | in der Quelle nicht angegeben | b | $\omega_{-1}$ | $R_{-1}$ |
| II | a | b | ω | $R_{-1}$ |
| III | b+1 | a | ω* | in der Quelle nicht angegeben (bei L=1 gelöscht) |
| IV | b+1 | a | ω* | R |

- **Wartezeiten:** $W_1$ liegt zwischen Phase I und Phase II (bis die Sektornummer von b unter dem Kopf steht), $W_2$ zwischen Phase III und Phase IV (Sektornummer von a). Beide sind Vielfache der Wortzeit. Im Knotengraphen sind das die Koinzidenztests an den Knoten 5/6 ($W_1$) sowie 26/27 und 57/64 ($W_2$).
- **Phase IV** kann mehrere Wortzeiten benötigen: je nach Befehlstyp wird die Operation 1- bis 32-mal ausgeführt, jede Ausführung kostet eine Wortzeit.
- **Entfall von $W_1$:** Endet ein GX- oder WX-Befehl unter den Bedingungen des X-Abbruchs, entfällt die Koinzidenzprüfung vor Phase II (Knoten 4, ANWEISUNG-002).
- **Befehlsentnahme nach X-Abbruch (Ergänzung 04.10.2026):** Ohne Wartezeit $W_1$ kommt der nächste Befehl nicht aus Zelle b+1, sondern aus der Spur des Befehlszählers und dem Sektor, der in der zweiten

Wortzeit nach dem Abbruch unter dem Kopf liegt (Sektor der Zusatzzelle plus zwei physische Schritte, je +15 oktal; Befehlsliste 4.1.6). AR und BZ bleiben b+1, nach Phase III des entnommenen Befehls ist BZ = b+2. Ein Programm muss den Folgebefehl daher in den fälligen Sektor legen (Beispiel: Oktaldruck, Programmieranleitung 5.17, Test `bsp_5_17_oktaldruck`). Umsetzung: ANWEISUNG-005 und -044 in "C8205-Interpreter-Beschreibung_der_Knoten.md", Kommentar am Knoten 78 in "C8205-Interpreter-Eingänge_der_Knoten.md".

- **Phase I und BZ (abgeleitet):** Liest man "gleichzeitig" wie in "C8205-Interpreter-Eingänge_der_Knoten.md" (Registerwerte vor dem Knoten), ergibt Phase I für den BZ den alten AR-Wert + 1 statt b+1. Das ist folgenlos, weil Phase II den BZ neu setzt.

### 4.2 Besonderheiten im Phasenablauf

Quelle: Programmieranleitung, Abschnitt 2.2.2 bis 2.2.4, 4.2.1 und 4.3.2.10.

- **Phase II:** Bei R-Befehlen wird der Adreßteil im BZ disjunktiv mit den Adreßstellen des AC verknüpft und der Operationsteil umgestellt. Bedingte Sprungbefehle werden bei erfüllter Bedingung in unbedingte umgestellt. Als bedingte Sprungbefehle gelten Negativsprung, Positivsprung, Überlaufsprung, Vergleichersprung 1, 2 und 3 sowie der bedingte Sprung mit Rückkehrabsicht.
- **Phase III:** Eine geforderte AC-Löschung geschieht hier. Bei einem Sprung mit Rückkehrabsicht wird der AC disjunktiv mit dem um 1 Bit rechtsverschobenen BZ verknüpft. Ein programmierter Stopp erfolgt am Ende der Phase III, die Fortsetzung hängt von der Bedienung ab. Eingabe-Halt- und Ausgabe-Halt-Befehle werden einer Sonderbehandlung unterworfen (die Programmieranleitung nennt dazu keine Einzelheiten). Zum Ausgabe-Halt gibt die Originaldokumentation (4.3.2.5, Abb. 21) an: Von der Stelle unmittelbar nach der Löschanweisung bis zum Beginn der Phase I vergehen mindestens zwei Wortzeiten; eine Umstellung der Operation nach Phase III in Überlaufsprung oder Vergleichersprung 3 ist bedeutungslos, weil die Sprungbedingung nur in Phase II ausgewertet werden kann (Befehlsliste, Ausgabe-/Anwahl-Befehl).
- **Nach Phase III:** Bei einem bedingten Sprung mit nicht erfüllter Bedingung folgt nach einer Wortzeit Phase I. Bei unbedingten Sprüngen und bedingten Sprüngen mit erfüllter Bedingung folgt die Resultatbehandlung in Phase IV, ebenso bei allen anderen Befehlen.
- **Resultatbehandlung:** Sie geschieht nach der Operation in Phase IV. Bei Organisationsbefehlen, die diesen Teil der Befehlsschleife nicht durchlaufen, ist sie wirkungslos, auch wenn die Z-Bits R≠0 enthalten (z. B. Leer-Befehl 164).
- **Markierte Einzelbefehle** erzeugen zusätzlich einen von drei Vergleichszuständen.
- **Überlauf:** Überlauf bedeutet, dass der Übertrag von $z_1$ nach $z_0$ vom Übertrag von $z_0$ zur nicht mehr vorhandenen Maschinenstelle verschieden ist. Er wird nur bei Addition, Subtraktion und bedingter Addition bzw. Subtraktion als Einzelbefehl oder als markierter Einzelbefehl ohne Resultatsbehandlung (R=0, d. h. Z=0 oder 1) eingeschaltet, sowie bei Befehlen, die in solche umgestellt werden. Ist im vorhergehenden Befehl ein Überlauf erfolgt und der abzuarbeitende Befehl kein Überlaufsprung, wird die Befehlsschleife vor Phase IV unterbrochen (im Knotengraphen: Halt wegen Überlauf, Lampe FÜ). Der Überlaufsprung wird dann bereits in Phase II in einen unbedingten Sprung umgestellt, gleichzeitig wird der Überlauf gelöscht.

## 5. Speicher-Timing und Leistungswerte

- Umdrehungsgeschwindigkeit der Trommel: 18 000 $min^{-1}$ = 300 $s^{-1}$
- Schreibdichte: 32 Wörter à 33 Bit je Spur
- daraus resultierende Arbeitsfrequenz der seriellen Verarbeitung: rund 316 kHz
- Wortzeit: 33 : 316 kHz ≈ 104 µs ≈ 0,1 ms (deckt sich mit der Phasendauer oben; entspricht zugleich der Zeit, in der ein gespeichertes Wort unter dem Lesekopf durchläuft)
- mittlere Zugriffszeit des Trommelspeichers: 1,67 ms (maximale Zugriffszeit: 3,33 ms)
- Adressierung: 12 Bit Adressteil des Befehlswortes → $2^{12}$ = 4096 adressierbare Speicherzellen; 9 Bit Operationsteil → $2^9$ = 512 mögliche Bitkombinationen für Operationen

Gesamtzeit je Befehlsabarbeitung (inklusive Stellen des Befehlszählers, Befehlsentnahme, Entschlüsselung der Operation, Aufstellen der Befehlsadresse und ggf. Wartezeiten):

| Befehlsart | min. Zeit | max. Op/s |
|---|---|---|
| Einzelbefehle | 0,5 ms (genauer 5 Wortzeiten = 0,52 ms) | 1900 |
| Organisationsbefehle | 0,4 ms (genauer 4 Wortzeiten = 0,416 ms) | 2400 |
| Gruppen-/Wiederholungsbefehle | — | bis ca. 9000 |

Aufgrund des halbanalytischen Aufbaus des Befehlskodes ist nicht jede der 512 möglichen Bitkombinationen sinnvoll belegt bzw. es gibt Kombinationen mit gleicher Wirkung; dadurch reduziert sich die Befehlsliste auf etwa 370 sinnvolle, in ihrer Wirkung voneinander verschiedene Befehle. Diese wiederum stellen Kombinationen von nur rund 20 Grundoperationen dar. (Die Befehle samt Formeln stehen in "C8205-Interpreter-Befehlsliste.md": generische Zusammenfassung nach Befehlsarten B=0…7 und Einzelbefehlsübersicht aller 512 Oktalcodes.)

## 5.1 Zeitoptimale Programmierung

Weil der Speicher eine rotierende Trommel ist (Interleaving: "C8205-Interpreter-Magnettrommel.md"), hängt die Laufzeit eines Programms davon ab, in welchem Sektor Befehle und Operanden liegen. **Zeitoptimal** heißt: Der Operand eines Befehls liegt so, dass er nach der Befehlsadressenaufstellung (Phase III) ohne zusätzliche Wartezeit unter dem Kopf ankommt. Die Befehle selbst legt man dazu fortlaufend ab (logisch nächster Befehl = physisch 5 Positionen weiter); optimiert wird die Lage der **Operanden** und der Sprungziele.

*Quelle:* Programmieranleitung, Abschnitt 5.19 "Zeitoptimale Programmierung" (Seitenabbild) und die Magnettrommel-Datei. Was aus diesen Quellen selbst berechnet ist und nicht im Wortlaut dort steht, ist mit "(abgeleitet)" gekennzeichnet.

**Positionen statt Sektornummern (abgeleitet).** Die Magnettrommel-Datei gibt den logischen Schritt +1 als +05 (oktal) in der physischen Reihenfolge an. Ein logischer Abstand d (in Sektoren) entspricht daher 5·d mod 32 Positionen (dezimal), jede Position eine Wortzeit (104 µs). Für die Abstände, die die Magnettrommel-Datei als zeitoptimal nennt:

| logischer Abstand zum Befehl (oktal, mod 40) | Positionen | Verwendung |
|---|---|---|
| +01 | 5 | nächster Befehl bei fortlaufendem Programm |
| +32 | 2 | Operand eines Einzelbefehls |
| +07 | 3 | Operand eines Einzelbefehls |
| +24 | 4 | Folgebefehl nach einem Sprung |

Der Operandensektor eines Befehls in Sektor s ist also (s+32) oder (s+07) mod 40 (oktal); die Spur spielt keine Rolle, da ein Bahnwechsel keine Zeit kostet. Die Magnettrommel-Datei nennt für Gruppenbefehle zusätzlich +24.

**Reihenfolge der Phasen und Wartezeiten.** Die Tabelle in Abschnitt 4 führt die Phasen in der Reihenfolge I–IV auf. Im zeitlichen Ablauf gehört Phase I (Befehlszähleraufstellung) bereits zum **nächsten** Befehl: die Reihenfolge je Befehl ist II, III, IV, I. Die beiden Wartezeiten heißen in der Programmieranleitung **$W_1$** (zwischen Phase I und der Entnahme des nächsten Befehls in Phase II) und **$W_2$** (zwischen Phase III und Phase IV, bis der Operandensektor erreicht ist).

**Mindestablauf (abgeleitet).** Der Ablauf ist in Positionen ab der Befehlsentnahme (Phase II = 0) gezählt:

| Befehl | Ablauf | Wortzeiten bis zur nächsten Befehlsentnahme |
|---|---|---|
| Einzelbefehl, Operand bei +32 | II 0, III 1, IV 2, I 3, $W_1$ 4, nächster Befehl 5 | 5 |
| Einzelbefehl, Operand bei +07 | II 0, III 1, $W_2$ 2, IV 3, I 4, nächster Befehl 5 | 5 |
| Sprung | II 0, III 1, IV 2, I 3, nächster Befehl 4 | 4 |

Das deckt sich mit Abschnitt 5 (Einzelbefehle mindestens 5 Wortzeiten, Organisationsbefehle mindestens 4 Wortzeiten).

**Beispiel 1: Operand nicht optimal (Programmieranleitung 5.19).** Der Befehl 7716 211 steht in Zelle 4000 (Sektor 00). Sein Operandensektor 16 ist weder +32 noch +07. Ablauf nach der Programmieranleitung: II in Sektor

00, III in 15, $W_2$ über 32, 07, 24 und 01, IV in 16, I in 33, dann $W_1$ von Sektor 10 bis Sektor 24 (der nächsten Umdrehung), nächster Befehl in Sektor 01. Von Befehl zu Befehl vergehen 37 Wortzeiten (abgeleitet: 4 Positionen $W_2$, 29 Positionen $W_1$), das ist eine Trommelumdrehung mehr als nötig.

**Beispiel 2: Sprung als Umweg.** Lässt sich der Operand nicht verlegen, setzt man einen unbedingten Sprung `xx24 000` in die Zelle (Sektor 00), verlegt den Lesebefehl in Sektor 24 und lässt ihn von dort aus auf den Operandensektor 16 zugreifen (24+32 mod 40 = 16). Der nächste Befehl folgt in Sektor 25. Von Befehl zu Befehl vergehen nur 9 Wortzeiten (Sprung 4, Einzelbefehl 5, abgeleitet). Die Programmieranleitung nennt eine Ersparnis von 28 Wortzeiten (37 − 9 = 28). Der Preis ist eine zusätzliche Zelle für den Sprung; der Befehl in Sektor 24 muss dafür frei sein.

**Beispiel 3: Algebraischer Ausdruck (Programmieranleitung 5.19.1, Aufgabe wie 5.5).** Transkription nach Seitenabbild; die Spalte "Abstand" ist abgeleitet (Positionen von der Befehlszelle zum Operanden- bzw. Sprungziel-Sektor, Magnettrommel-Tabelle):

| Adr. | Inhalt | Bemerkung | Abstand |
|---|---|---|---|
| 3003 | 3006 000 | Sprung zum Programmstart | 15 |
| 3006 | 3000 215 | ⟨Ac⟩ := 2a | 2 |
| 3007 | 3001 220 | ⟨Ac⟩ := 2a − b | 2 |
| 3010 | 3002 224 | ⟨Ac⟩ := 2·(2a − b − c) | 2 |
| 3011 | 3031 000 | Sprung | 16 |
| 3031 | 3000 214 | ⟨Ac⟩ := 2·2·(2a − b − c) + a | 3 |
| 3032 | 3001 220 | ⟨Ac⟩ := x | 3 |

Alle Rechenbefehle entnehmen ihren Operanden zeitoptimal (2 bzw. 3 Positionen). Die beiden Sprünge sind dagegen **nicht** wartefrei (wartefrei wäre +24, also 4 Positionen; von 3011 aus 3035). Das ist offenbar beabsichtigt: Die Sprungziele sind durch die Operandenlage bestimmt, denn Zelle 3000 ist nur von Sektor 06 (+32) oder Sektor 31 (+07) aus wartefrei erreichbar. Nach der Programmieranleitung vergeht von der Entnahme des ersten Befehls bis zur Entnahme des letzten Operanden "genau eine Trommelumdrehung" weniger als in 5.5 (dort ca. 2⅔ Umdrehungen); diese Angabe ist hier nicht nachgerechnet.

**Folgerungen für die Programmierung (abgeleitet).**

- Befehle fortlaufend ablegen, Operanden (Variablen, Konstanten, Adreßwörter) auf Sektor +32 oder +07 zum benutzenden Befehl legen. Mit `equ` und `db` lassen sich die dafür nötigen Lücken einstellen; die Lücken können mit Daten gefüllt werden.
- Ein Operand, der von mehreren Befehlen benutzt wird, ist nur für Befehle in passenden Sektoren optimal erreichbar. Sonst entscheidet man je Befehl zwischen Wartezeit und Umweg über einen Sprung.
- Ein Sprung hat sein wartefreies Ziel bei +24. Jedes andere Ziel kostet Wartezeit.

**Hinweis zum Interpreter.** Der Python-Interpreter rechnet mit Wortzeit und Wartezeit = 0 (Abschnitt 4; Ausnahme: die Befehlsentnahme nach X-Abbruch, Abschnitt 4.1); die Zeitangaben dieses Abschnitts beschreiben die Hardware. Um sie am Interpreter zu prüfen, müsste dieser die Trommelposition (Wortzeitzähler mod 32 und Reihenfolge der Magnettrommel-Datei) mitführen.

**Tabellen der zeitoptimalen Sektoren.** Die Nachschlagetabellen der Programmieranleitung 5.19 (Spalten A bis F) stehen, neu berechnet, in "C8205-Interpreter-Befehlsliste.md", Abschnitt "Tabellen der zeitoptimalen Sektoren".

# 6. Teilprojekte

## 6.1 Interpreter

Der Interpreter (`c8205_interp.py`) führt Programme des C 8205 interpretativ aus: Er läuft den Knotengraphen des Befehlsablaufs (Phasen I bis IV) ab und bildet Register, Flags, Trommelspeicher und Peripherie nach (alle Zeiten = 0, bis auf die Befehlsentnahme nach X-Abbruch, Abschnitt 4.1). Testaufträge liegen in `Tests/`, Protokolle in `Test-Protokolle/`, der Trommelspeicher als persistente Datei in `Trommel/` (er behält seinen Inhalt über Tests hinweg). Der Vollständigkeitstest `c8205_befehlssweep.py` prüft alle 512 Operationscodes gegen den Knotengraphen. Einzelheiten: "C8205-Interpreter-Python_Interpreter.md" (Bedienung, Schalter, Geräte), "C8205-Interpreter-Durchführung_eines_Tests.md" (Testdatei, Tastenfolge, Protokoll) und "C8205-Interpreter-Lochstreifen-Hilfsprogramme.md" (Hilfsprogramme für virtuelle Lochstreifen).

**GRAFIK:** Der Interpreter hat zwei Betriebsarten, TEST (Stapelbetrieb) und GRAFIK (Pult- und Schreibwerk-Fenster, Töne, echte Trommel). Die Regeln des Rechners sind für beide gleich; Beschreibung: `C8205-Interpreter-GRAFIK_Beschreibung.md`.

### 6.2 Assembler

Der Cross-Assembler `c82ass` (C, Unterverzeichnis `Assembler/`) übersetzt Assembler-Quelltext (`.asm`) in ladefähigen Objektcode für den C 8205. Der Objektcode ist für das "Kleine Eingabeprogramm" des C 8205 bestimmt (Bedienpult-Dokument, Abschnitt 4.5: ein einfaches Leseprogramm auf Spur 0, das nur die Zeichen 0, 1, Punkt, Doppelpunkt und Zwischenraum verarbeitet); er besteht daher aus Zeilen im K1K2-Format. Zusätzlich erzeugt der Assembler eine Trommeldatei im Testformat des Interpreters (`.test`, mit Platzhalter-Definitionen für Testmarken und Tastenfolge), aus der sich ein Testauftrag zur Prüfung eines assemblierten Programms im Interpreter ergänzen lässt.

Einzelheiten stehen nicht hier, sondern an einer Stelle:

- Aufruf, Sprache, Anweisungen, Pässe, Makros, Beispiel: `Assembler/Dokumentation/C8205_Assembler_Dokumentation.odt` (maßgeblich; die `.md`-Datei ist nur eine durchsuchbare Ableitung),
- Verzeichnisse und vom Assembler erzeugte Dateien: "C8205-Projektstruktur.md", Abschnitt 3,
- vom Assemblerergebnis zum Testauftrag des Interpreters: "C8205-Interpreter-Durchführung_eines_Tests.md", Abschnitt 2a.

### 6.3 BIS (Bausteininterpretiersystem)

Das BIS ist ein offenes Unterprogrammsystem für den C 8205 (Ein- und Ausgabe von Dezimalzahlen, Multiplikation und Division von Fest- und Gleitkommazahlen, Addition und Subtraktion von Gleitkommazahlen, Serviceprogramme). Es ist Software für den Rechner und kein Teil der simulierten Hardware; Interpreter und Assembler kennen es nicht. Die Hardware kennt nur die Eingabe eines Zeichens, alles Weitere leisten die Bausteine des BIS. Zum BIS gehören das "Kleine Eingabeprogramm" (KEP, Spur 0, Adressen 0000 bis 0037 oktal), das der Assembler-Objektcode (6.2) voraussetzt, und das Minimalprogramm. Erfasst sind bisher die allgemeine Beschreibung, das KEP und das Minimalprogramm.

Einzelheiten stehen nicht hier, sondern in "C8205-Interpreter-BIS.md"; Einordnung als Teilprojekt: "C8205-Interpreter-Projektueberblick.md", Abschnitt 3a, und "C8205-Projektstruktur.md", Abschnitt 3a.

## 7. Verweise auf die Teildokumente

**Rechner und Regelwerk**

- Trommelorganisation im Detail (128 Spuren/32 Sektoren, Interleaving/logische vs. physische Sektoradressen, Bahnwahl, Bahnsperrenschalter): "C8205-Interpreter-Magnettrommel.md". Anwendung auf die Programmierung (zeitoptimale Operanden- und Sprungziel-Lage): Abschnitt 5.1 dieser Datei; Tabellen der zeitoptimalen Sektoren: "C8205-Interpreter-Befehlsliste.md".
- Register-, Feld- und Flag-Definitionen im Detail (AC, OR, BZ, AR, BR, Maschinenwortaufbau, Flags E, A, T, SW, K1–K3, FE, FÜ, BS, US, V1, V2, H, SP): "C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md".
- Bedienelemente im Detail (Schalter H-R, Tasten E/G/M/Ü/SW/ST, Eingabetasten, Kanalauswahl, Selektoren SP/BS, Schalter LS, Kleines Eingabeprogramm): "C8205-Interpreter-Bedienpult_mit_Schaltern.md".
- Lochstreifenleser, Lochstreifenstanzer, Schreibwerk-Ausgabe (Hardware): "C8205-Interpreter-Lochstreifenleser.md".
- Zeichenkode (Tabelle 1, R300-Lochstreifenkode): "C8205-Interpreter-Kodetabellen.md".
- Befehle und Formeln: "C8205-Interpreter-Befehlsliste.md".
- Ablaufdetails der Befehlsverarbeitung als Knoten-/Testgraph: "C8205-Interpreter-Eingänge_der_Knoten.md" und "C8205-Interpreter-Beschreibung_der_Knoten.md" (Variablentabelle T2).

**Interpreter und Tests**

- Python-Interpreter (Aufbau, Aufruf, Schalter, Geräte, Protokoll): "C8205-Interpreter-Python_Interpreter.md".
- Testdurchführung (Testdatei, Tastenfolge, Ergebnisprüfung): "C8205-Interpreter-Durchführung_eines_Tests.md".
- Lochstreifen-Hilfsprogramme (Anzeige, Eingabe und Umwandlung virtueller Lochstreifen): "C8205-Interpreter-Lochstreifen-Hilfsprogramme.md".

- Cache der BAZ-Wirkungen (für den Claude-Interpreter; der Python-Interpreter liest ihn nicht): "baz-wirkungstabelle.md".
- Einführung in das Gesamtprojekt für Außenstehende: "C8205-Interpreter-Projektueberblick.md".
- Einstieg für Neueinsteiger (Lesereihenfolge, wo man bei einer Frage weiterliest): "C8205-Einstieg_fuer_Neueinsteiger.md".
- Dokumentenlandkarte (alle .md-Dateien mit ihrer Rolle): "C8205-Interpreter-Projektueberblick.md", Abschnitt 6.

**Assembler**

- Teilprojekt, Verzeichnisse, erzeugte Dateien: "C8205-Projektstruktur.md", Abschnitt 3.
- Sprache, Anweisungen, Makros, Beispiel: "C8205_Assembler_Dokumentation.odt" (maßgeblich) bzw. ".md" (Ableitung).
- Vom Assemblerergebnis zum Testauftrag des Interpreters: "C8205-Interpreter-Durchführung_eines_Tests.md", Abschnitt 2a.

**BIS**

- Inhalt (allgemeine Beschreibung, KEP, Minimalprogramm, Eingabekennzeichen $K_1$/$K_2$): "C8205-Interpreter-BIS.md".
- Einordnung als Teilprojekt: "C8205-Interpreter-Projektueberblick.md", Abschnitt 3a; "C8205-Projektstruktur.md", Abschnitt 3a.
- Zahlendarstellungen: Übersicht, ganze Festkommazahl und interne Gleitkommadarstellung: "C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md"; externe Gleitkommadarstellung und Tetraden des BIS: "C8205-Interpreter-BIS.md", Abschnitt BIS-1.1.

**Ausgabe**

- Gestaltung aller PDF-Ausgaben sowie Namensschema und Liste der erzeugten PDFs: "C8205-Interpreter-PDF.md" (Abschnitt 8: Dateibenennung).
- Verzeichnisstruktur des Gesamtprojekts: "C8205-Projektstruktur.md".
- Versionsnummern der Dateien (Aufbau `y.xx-zzz`, Regeln, Ort der Nummer): "C8205-Versionierung.md".

## 8. Offene Punkte

Keine offenen Punkte.